NLP1 Tutorial 1: Concepts and Questions

Nov 6, 2026 · @Isha

The tutorial builds a text classifier in five steps: store text as Unicode, cut it into tokens (BPE), turn tokens into a vector, map the vector to class probabilities (softmax regression or an MLP), and compare systems with Bayesian estimation. Every "Questions" box in the tutorial is answered below; the oral's follow-up questions can come from any of them.

Digital text: Unicode and encodings

Inside Python, text is a sequence of Unicode code points; on disk it is bytes in some encoding. You must know a file's encoding to read it back correctly.

- **Unicode** is one big table that gives every character of every script (plus emoji) a number, its **code point** (e.g. é = U+00E9).
- An **encoding** says how code points become bytes:

Encoding	Bytes per character	Covers
ASCII	1 (7 bits used)	128 characters: English letters, digits, punctuation
ISO-8859-1 (Latin-1)	1	256 characters: adds Western European accents (é, ç)
UTF-8	1 to 4	all of Unicode; the first 128 match ASCII

Reading with the wrong encoding either fails loudly or corrupts silently:

- Latin-1 bytes read as UTF-8 → an error (`UnicodeDecodeError`: byte 0xe9 is not valid UTF-8 there).
- UTF-8 bytes read as Latin-1 → no error, but garbled text (*mojibake*): "Café" becomes "CafÃ©". Every multi-byte character turns into 2–4 wrong ones.

In our project: 26% of bios contain non-ASCII characters, almost all typographic (curly quotes ’ “ ”, dashes, é). Our tokenizer applies **NFKC normalisation**, which maps equivalent forms of a character (e.g. a ligature "fi" and "fi") to one form.

Likely questions

- **What is the difference between a character, a code point and a byte?** A character is what you see; a code point is its number in Unicode; bytes are how an encoding stores that number.

- **Why might `len(text)` surprise you?** It counts code points, not bytes and not always visible characters (an accent can be a separate combining code point).
- **Why is a wrong encoding that gives no error worse than one that crashes?** The corruption goes unnoticed and enters your features as junk tokens.

Text segmentation and words

There is no single right unit for cutting text: it depends on how you count and on the language. The sentence "They picnicked by the pool, then lay back on the grass and looked at the stars." has 16 tokens if you split on spaces ("pool," and "stars." keep their punctuation), 18 if you split punctuation off, and 16 again if you delete punctuation, but those 16 are different from the first 16.

Written Chinese has no spaces. 姚明进入总决赛 ("Yao Ming reaches the finals") can be read as 7 characters, 3 words (姚明 · 进入 · 总决赛) or 5 units: there is no orthographic word to rely on.

Linguists' units:

Unit	Meaning	Example
orthographic word	a string between spaces or punctuation	"cats"
morpheme	the smallest meaningful part, memorised by heart	`cat` + `-s`; `cata-` + `strophe` + `-ic` in *catastrophic* (no `cat` in it!)
word (linguist's sense)	built from morphemes by *morphology*, then combined into sentences by *syntax*	*cats*
listeme	anything that must be memorised because its meaning cannot be derived from its parts	*kick the bucket* (= to die)

Morphemes would be ideal units: **meaningful, conventional** (shared by speakers) and **productive** (they combine into new words). But no script marks morpheme boundaries, and many scripts do not even mark word boundaries (*scriptio continua*). That is why the tutorial turns to a **data-driven** solution: BPE learns units that only need to be **productive** (reusable), not meaningful.

The tutorial's questions

1. **Shortcomings of treating European-language text as a sequence of characters?** A single character carries almost no meaning, sequences become long, and a model must learn to build meaning from scratch, which needs much more data and compute. A *bag* of characters throws away nearly everything. (Upsides: a tiny vocabulary and no unknown words.)
2. **Would English word-boundary rules work for other European languages?** Only partly. Spaces help everywhere, but German and Dutch glue nouns into long compounds (*Arbeitsunfähigkeitsbescheinigung*), French and Italian attach clitics with apostrophes (*l'homme*, *dell'arte*), and Finnish, Hungarian or Turkish build long words from many suffixes. Each language needs its own rules.
3. **Would the vocabulary of observed words be similar in size across languages?** No. Morphologically rich languages produce many more distinct word forms for the same amount of text (Finnish has many case endings per noun; English has few), so their vocabularies grow much faster.

Heaps' law

The more text you read, the more new words you keep meeting; the vocabulary never stops growing, it just grows more slowly. With n = number of tokens read and V(n) = number of distinct tokens (types) seen so far:

```
V(n) = K \, n^{\beta} \qquad\Longleftrightarrow\qquad \log V(n) = \log K + \beta \log n
```

On log–log axes this is a straight line with slope β. K and β depend on the language and the dataset; for English, β is typically around 0.4–0.6 (below 1: growth slows but never stops).

How to verify it (the suggested exercise): take a corpus (e.g. AG News), tokenize it with a linguistically informed tokenizer (NLTK, not BPE, because BPE's vocabulary is capped by design), walk through the tokens, and record after each token how many tokens and how many distinct types you have seen. Plot both on log–log axes and fit a line: the slope is β, the intercept is log K.

Why it matters: with word-level units, a test text will always contain words never seen in training (unknown words), and the feature space keeps growing with more data. Subword units (BPE) fix both: a fixed vocabulary size, and any new word can be written with known pieces.

Likely questions

- **What would β = 1 mean?** Every token is a new type: no repetition at all. β = 0 would mean a fixed, closed vocabulary.

- **Why should you not use BPE to check Heaps' law?** BPE's vocabulary has a maximum size by construction, so V(n) flattens at that size instead of growing.
- **How does this relate to Zipf's law?** Zipf describes word *frequencies* (a few words are very frequent, most are rare); that long tail of rare words is why new types keep appearing.

Byte-pair encoding (BPE)

BPE learns a fixed-size vocabulary of subword units from data: start from characters and keep gluing together the most frequent neighbouring pair. The units only need to be *productive* (reusable), not meaningful like morphemes (Sennrich et al., 2016).

Learning the vocabulary (fitting)

1. **Pre-tokenize:** split the corpus into rough pieces (the tutorial splits on whitespace and marks word boundaries with a special `_` character). BPE never merges across these pieces.
2. **Initial vocabulary:** all characters seen, plus an unknown symbol (`⁇` in the demo, `[UNK]` in Hugging Face) for characters never seen.
3. **Count pieces:** how often each pre-token occurs, written as a sequence of characters.
4. **Count pairs:** for every neighbouring pair of units, add up the counts of the pieces it occurs in.
5. **Merge the most frequent pair** (ties broken alphabetically), add it to the vocabulary, record the merge and its order, and rewrite the pieces with the merged unit.
6. **Repeat** until the vocabulary size is reached, a maximum number of merges is done, no pair occurs at least `min_frequency` times, or nothing is left to merge.

Demo corpus "low lower newest widest / low lowest newer wider": the pair `l` + `o` occurs 4 times (low twice, lower, lowest), so one of the first merges creates `lo`, later `low`, and so on.

Tokenizing new text: pre-tokenize, split into characters, then apply the recorded merges **in the order they were learned**. Unseen characters become the unknown symbol. Example from the demo: "newer löwell" gives familiar pieces for the known parts and `⁇` for ö.

In the experiment the tutorial switches to the Hugging Face `tokenizers` library (well tested, fast) with three hyperparameters: `vocab_size` (how many units, related to the number of merges), `min_frequency`, and the pre-tokenizer (`Metaspace`). It trains on AG News with `vocab_size=1000`, `min_frequency=2`.

The tutorial's questions

1. **Effect of the hyperparameters on the units?** A larger vocabulary or more merges gives longer units: frequent words become whole tokens, and a text needs fewer tokens. A smaller one leaves mostly characters and common fragments. A higher minimum frequency stops merges of rare pairs, so fewer, more frequent units. The change is predictable: the most frequent strings are merged first.
2. **How does this affect how informative tokens are?** Longer units (whole words like *surgeon*) carry far more meaning than single characters. But a very large vocabulary makes many units rare, so a model sees each one only a few times and learns poor weights for it. It is a trade-off between informative units and enough examples per unit.
3. **How do lowercasing and pre-tokenization affect this?** Lowercasing merges *He*/*he*, freeing vocabulary slots for more words, but loses case information (names, acronyms, ALL-CAPS headings). Splitting punctuation off stops "engineer." and "engineer" from becoming different units. The pre-tokenizer also decides what can *never* merge: splitting on spaces means no unit spans two words. Unicode normalisation (e.g. NFKC) makes equivalent characters identical before counting.

Likely follow-ups

- **Why does the order of merges matter when tokenizing?** Applying merges in a different order can give a different segmentation; replaying the training order gives the same segmentation the vocabulary was built from.
- **Can BPE produce an unknown token for a new word?** Not for a new word made of known characters, since it can always fall back to single characters. Only characters never seen in training become unknown.
- **Is BPE language-independent?** Mostly: it needs no rules, only data. But the pre-tokenizer assumes spaces; for Chinese you would pre-tokenize per character.

Text classification: model + decision rule

A text classifier = a probabilistic model that maps a text x to a probability for each of the K classes, plus a decision rule that turns those probabilities into one class.

```
text x  →  probabilistic model  →  p(1|x), …, p(K|x)  →  decision rule  →
predicted class ŷ
```

- The output of the model is a **pmf** (probability mass function): K non-negative numbers that sum to 1, a point on the *probability simplex*. Strictly, the model *prescribes* or *parameterises* the pmf; the *system* predicts a class.

- **Tabular view:** you can imagine the model as a giant table with one row per possible text and one column per class (a *tabular CPD*, conditional probability distribution). Looking up row x gives its pmf. It is a useful metaphor but useless in practice (see the questions).
- **Decision rule:** the usual one is **most probable class**: $\hat{y} = \text{argmax}_k\ p(k|x)$, the mode of the pmf. It is a convention, not a law; alternatives minimise expected cost (Bishop, section 1.5).
- In code: `NamedClasses` maps class names to integers 0…K−1; `TextClassifier(outcome_space, model, decision_rule)` chains the pieces.

The tutorial's questions

1. **How many parameters does a literal table need?** One row per possible text, each with K − 1 free numbers (the last is fixed because a row sums to 1): |X| × (K − 1). The set of possible texts is unbounded (countably infinite), so infinitely many parameters. Even limited to texts of at most L tokens from a vocabulary of V, that is about V^L rows.
2. **Would an estimated table be sparse or dense?** Extremely sparse. Almost every text occurs once or never (Zipf, Heaps), so most rows have no data at all, and observed rows are mostly a single 1 (the one label seen) and 0s elsewhere.
3. **Can the table classify a new, never-seen text?** No. There is no row for it, so the lookup fails (or you fall back to a default such as the class prior). The table cannot generalise.
4. **Why do tables not get us far in NLP?** Language is productive and open-ended: new texts are created all the time, and nearly every text is unique. A table treats every text as unrelated to every other, so it cannot share what it learned from similar texts ("a great day" tells it nothing about "a lovely day"). We need a function of *features* of the text instead.
5. **When does the most-probable-class rule go against intuition?** (a) Nearly flat pmfs: p = (0.34, 0.33, 0.33) picks class 1 although it is wrong with probability 0.66. (b) Unequal costs: p(benign) = 0.6, p(tumour) = 0.4 picks "benign", but missing a tumour is far worse, so minimising expected cost would flag it. (c) In our project: for an ambiguous academic bio, argmax picks "professor", the big class, every time, so smaller classes like psychologist lose out.

Text representation: encoders and named axes

An encoder turns a text into a point h(x) in a fixed D-dimensional space, so that a regressor can map that point to probabilities. Text is not a point by nature; making it one requires assumptions, and the encoder is where those assumptions live.

```
text x  →  encoder  →  h(x) (D numbers)  →  softmax regression  →  p(1|x), …, p(K|x)
```

- **Named axes:** give each dimension a meaning, e.g. dimension 1 = "*good* occurs", dimension 2 = "*not … good* occurs". The toy encoder uses `split()` plus lowercasing and two axes (*good*, *bad*), so "a GOOD day" is caught but "a goooood day" is not.
- **Token indicators:** one axis per vocabulary token, value 1 if it occurs, else 0 (the tutorial's `TokenIndicator`).
- **Bag-of-words:** one axis per token, value = how many times it occurs. The name: write the text on paper, cut it into tokens, throw them in a bag. Order is gone.
- **Bag-of-bigrams:** one axis per *pair* of neighbouring tokens.
- In this tutorial the encoder is fixed preprocessing; in later weeks it becomes trainable.

The tutorial's questions (for a count space with one dimension per token of a vocabulary of size V)

1. **Dimensionality of h(x)?** V.
2. **Does it depend on x, e.g. on its length?** No, it is always V. Length changes the *values* (the counts add up to the number of tokens), not the number of dimensions.
3. **Can it tell "man bites dog" from "dog bites man"?** No: both have one *man*, one *bites*, one *dog*, so the same vector.
4. **Would a BPE tokenizer change that?** No. BPE changes which units are counted, but it is still a bag: the same pieces in a different order give the same counts.
5. **Same questions for a space of adjacent token pairs (bigrams):** the dimension is up to V^2 (one axis per possible pair, in practice the pairs actually observed), still independent of x. It *can* separate the two sentences: (man, bites) and (bites, dog) versus (dog, bites) and (bites, man). With BPE, the same holds for pairs of pieces. It keeps local order, but longer reorderings can still collide.

Likely follow-ups

- **Indicators vs counts: which is better?** Counts add emphasis (a topic word repeated 5 times), indicators are robust to repetition and length. For short texts they are nearly identical: in our bios 78% of counts are 1.
- **What is the cost of bigrams?** Many more, much rarer features; usually needs a frequency cut-off or hashing.
- **Why sparse vectors?** A text uses a few dozen of V dimensions; storing only non-zeros (`scipy.sparse.csr_matrix`) saves memory and time.

Log-linear softmax regression

Each class k gets a weight vector w_k and a bias b_k; its score is a weighted sum of the features, and softmax turns the K scores into probabilities. With two classes this is logistic regression; scikit-learn calls the K-class version `LogisticRegression` too.

```
s_k(x) = \mathbf{w}_k^\top \mathbf{h}(x) + b_k, \qquad p(k \mid x) = \frac{\exp(s_k(x))}{\sum_{i=1}^{K} \exp(s_i(x))}
```

- **Log-linear:** the features and the parameters interact linearly in the scores (the *logits*), and the log of each probability is linear in them up to a normalising term.
- **Parameters:** a K × D weight matrix W and K biases b, so p(·|x) = softmax(W h(x) + b).
- **Estimation:** maximum likelihood (make the observed labels probable), with a gradient-based optimiser.
- **Toy example:** encoder axes (*good*, *bad*), W = [[−1, 1], [0, 0], [1, −1]] for (negative, neutral, positive): "a good day" leans positive, "a bad day" negative, "a regular day" gets all-zero features and so a uniform pmf.

The tutorial's questions

1. **What design difference lets it handle any text, seen or not?** The table had a separate parameter per text. Here the parameters belong to **features** that texts share, and the pmf is computed by a fixed formula from those features. Any new text maps to a feature vector, and so to a pmf. Shared parameters are what let it generalise.
2. **Why can even the best parameters give uninformed or wrong pmfs?** The model can only be as good as its representation and its form. If h(x) cannot distinguish two texts ("not good" and "good" share the *good* feature in a bag-of-words), they get similar pmfs no matter the weights. A linear model also cannot express interactions ("good" flips meaning after "not"). And a text with no known features ("a strange day") gets an uninformed pmf. "Best" means best within this model family on this data, not the truth.

Likely follow-ups

- **Why softmax?** It maps any K real numbers to a valid pmf: positive and summing to 1. The largest score gets the largest probability.
- **Is training convex?** Yes, for the log-linear model: one global optimum, so no dependence on initialisation.
- **What does the solver warning in the tutorial mean?** `saga` stopped at `max_iter=100` before fully converging; the weights are close to, but not exactly at, the optimum.

The experiment: AG News topic classification

The tutorial classifies news articles into 4 topics (world, sports, business, sci/tech) with the full pipeline, then improves it in two ways: counts instead of indicators, and an MLP instead of the log-linear model.

Step	What the tutorial does	Notes
Data	AG News, about 120k training articles, 4 classes of equal size	balanced, so accuracy is reasonable here
Tokenizer	Hugging Face BPE, `vocab_size=1000`, `min_frequency=2`, `Metaspace` pre-tokenizer, trained on the AG News training texts	`tokenizer.encode(text)` gives `.tokens` and `.ids`
Splits	`cross_validation_split`: shuffle with a seed, cut into k = 5 folds; the test set stays unseen	not stratified; several seeds = repeated CV
Encoder	`TokenIndicator`: a vector of vocabulary size, 1 where a token occurs	built as a dense numpy vector, then made sparse
Log-linear model	scikit-learn `LogisticRegression(solver="saga")` on a `csr_matrix`, `max_iter=100`	"don't mind the convergence warning"
Evaluation	accuracy per fold; `plot_box_and_points` shows all fold scores	about 3 minutes per 5-fold run
Bag-of-words exercise	counts instead of presence	a small accuracy increase
MLP	`SparseMLP`: input → 128 hidden (ReLU, dropout 0.1) → 4, `LogSoftmax` + `NLLLoss`, AdamW (lr 1e-4, weight decay 1e-4), batch 64, 10 epochs	about 20 minutes per 5-fold run on a CPU

The MLP in a sentence: a linear layer to H hidden units, ReLU, a linear layer to K scores, softmax. It is the log-linear model with one non-linear layer in between. The first layer multiplies a *sparse* input with a dense weight matrix (`torch.sparse.mm`), so only the non-zero tokens cost anything. `LogSoftmax` is used instead of softmax for numerical stability (it outputs log-probabilities; `NLLLoss` on top equals cross-entropy).

The tutorial's questions

1. **Is accuracy a good metric here? Justify with a property of the dataset.** For AG News, yes: its 4 classes are equally frequent, so accuracy is not dominated by one class and the majority baseline is only 25%. On an imbalanced dataset (like ours, 30% professor) it would be misleading.
2. **An alternative?** Macro-F1 (average F1 per class), balanced accuracy (average recall per class), per-class precision and recall, or a confusion matrix. To judge probabilities rather than decisions: log-likelihood of the true labels.
3. **Why might bag-of-words beat indicators?** Counts add information: a topic word repeated several times ("election", "election", "election") signals the topic more strongly than a word mentioned once in passing.

Likely follow-ups about the code

- **Why `csr_matrix`?** Each article uses a handful of the vocabulary, so storing only non-zeros saves memory and makes the multiplications fast.
- **Why Xavier initialisation (the `# why xavier?` comment in `smlp.py`)?** It draws starting weights with a variance scaled by the layer's input and output sizes, so the signal neither explodes nor dies out as it passes through layers.
- **What does dropout do at test time?** Nothing: `model.eval()` switches it off; it only acts during training.
- **What is questionable in `train_neural_network_model`?** It picks the best epoch by validation loss and then reports that same validation fold's accuracy, which is slightly optimistic. A separate hold-out from the training data avoids this (we did that in our project).
- **Why repeat cross-validation with several seeds?** The average over folds is itself random; repeating with different shuffles gives more scores and a better picture of the variation.

Comparing systems: Bayesian estimation

The average score over folds is itself random (change the seed or the number of folds and it changes), so to decide between two systems we estimate how big their difference plausibly is, and how sure we are, instead of running a significance test.

Why not a significance test (NHST)? The tutorial's position: statistics should *represent and communicate uncertainty*, because that is what a decision maker needs. A p-value answers "could the difference be exactly zero?" and washes the uncertainty away; it says nothing about how large the difference is. The key word is **estimation** (versus *testing*); being Bayesian is just what makes it possible to put probabilities on unknowns.

The data analysis model (Benavoli et al., 2017, section 3: the "Bayesian correlated t-test")

- **Data:** the per-fold differences m_i = score_B,i − score_A,i, on the same folds.
- **Likelihood:** the differences are drawn from a multivariate normal with the same unknown mean μ (the true mean difference) and a covariance with variance σ² on the diagonal and ρσ² elsewhere.
- **ρ (correlation between folds)** is not learned; it is set by a heuristic, ρ = n_test / n_total (= 1/k for k folds), because folds share training data (Nadeau & Bengio, 2003).
- **Priors:** a Student-t prior on μ (heavy tails, robust to differences far from 0) and a uniform prior on σ.
- **Posterior:** for a single dataset the posterior on μ is itself a Student-t, so no sampling is needed.
- The tutorial authors note the name "t-test" is misleading: nothing is tested; it is estimation.

```
prior belief about μ  +  measured fold differences  →  posterior: how
believable each value of μ is
```

Summarising the posterior for a decision

- **ROPE (region of practical equivalence):** a band around 0 of differences too small to be worth the effort. The size is a domain judgement, not something statistics can choose. The tutorial uses **0.005 accuracy**, because NLP papers usually dismiss gains under half a point.
- **Three probabilities:** the posterior area left of the ROPE = P(A better), inside = P(practically equal), right = P(B better). `BenavoliCorrelatedT(sys_a, sys_b).probs(rope)` returns them; the difference is B − A.
- **Plot:** the posterior curve, the shaded ROPE, a 95% HDI bar (the narrowest interval holding 95% of the belief) and the median.
- **Effect size:** posterior mean difference / posterior standard deviation (a standardised mean difference). Not in accuracy units; larger means the two systems' underlying means overlap less. It is "an inference worth reporting", not a certificate.
- **How to decide:** statistics does not decide for you. It tells you how much belief lands in each region given your assumptions; you combine that with practical concerns (cost, simplicity).

Likely follow-ups

- **What changes with more folds or more seeds?** More measurements narrow the posterior, but less than you might hope: the variance is multiplied by 1/n + 1/(k−1) (n = number of scores, k = folds). Extra seeds shrink only the 1/n part; extra folds shrink 1/(k−1) but make each test fold smaller and its score noisier.

- **Why is ignoring the correlation dangerous?** The posterior would be too narrow: you would be over-confident that a difference is real.
- **What does "practically equal with probability 0.97" mean?** Given the data and the model, there is a 97% chance that the true mean difference lies within the ROPE.
- **How did we choose our ROPE?** ±0.01 macro-F1: a difference we would act on, about 3× our fold-to-fold noise, slightly wider than the tutorial's because macro-F1 swings more; checked with 0.005 and 0.02.

System combination (ensembles)

Cross-validation leaves you with many trained copies of a design (seeds × folds); instead of picking one, you can combine them into an ensemble.

Method	How it combines M models for a text x	Note
Majority vote	each model predicts a class; the most common class wins	the tutorial's `MajorityVoteClassifier`
Mixture of experts (interpolation)	average the M pmfs, then pick the most probable class	smooth; a confident model counts more
Product of experts	average the M sets of logits (scores), then apply softmax	classes must be supported by all models

- Log-linear models can sometimes be combined in theory (e.g. averaging weights), but that does not carry over to neural models, which is why the tutorial uses ensembles.
- `WrapperForLoglinearModel` / `WrapperForNeuralModel` make saved sklearn or torch models look like the tutorial's `ProbabilisticModel` interface, and `ensemble_loglin` loads all of them.
- **The devtest point:** choosing the ensembling strategy by looking at the test set uses the test set for a decision, which makes its score optimistic. Properly you need a separate **devtest** split for such choices. (In our project, `dev` could play that role later.)

Likely follow-ups

- **Why does an ensemble usually help?** Different copies make partly different mistakes; voting or averaging cancels some of them and reduces variance.
- **When would it not help?** When all copies make the same mistakes (same data, same bias), e.g. our gender-correlated confusions would likely survive ensembling.

The final seven questions

These close the tutorial and are the closest to what a follow-up question looks like. Each answer links back to what we saw in our own project.

1. **Use Unicode characters as units instead of BPE: a good idea?** Mostly no, for these classifiers. Pros: a tiny vocabulary, no unknown tokens, robust to typos. Cons: a single character carries almost no meaning, so a bag of characters throws away nearly everything ("dog" and "god" become identical); sequences get very long; a model would need to learn to build words from scratch. BPE sits in between: frequent words whole, rare ones in pieces.
2. **Is a larger vocabulary necessarily better?** No. Larger means longer, more meaningful units, but each unit is rarer, so its weight is learned from fewer examples; the feature space and the model grow (our MLP's first layer is vocabulary × hidden); and returns diminish once common words are whole. The best size depends on how much data you have. We chose 10,000, which made 21 of 28 job names single tokens.
3. **Would a linguistically informed tokenizer (e.g. NLTK) do better or worse?** It depends. Pros: whole, interpretable words. Cons: an open vocabulary that keeps growing (Heaps' law), unknown words at test time, and variants (*nurse*, *nurses*, *nursing*) as unrelated features unless you stem. BPE fixes the size and shares pieces between variants. With lots of data, word-level bag-of-words can be competitive; it is an empirical question.
4. **Tokenizer trained on Europarl, classifier on AG News: what happens?** A domain mismatch. The merges reflect parliamentary language, so news-specific words (team names, tech brands) are split into many small, less informative pieces, texts become longer token sequences, and useful words never become single features. It still runs (no unknown words thanks to character fallback), but accuracy likely drops. This is why we trained our tokenizer on the bios themselves.
5. **Two ideas to improve the representations, and why you believe in them?** (a) **TF-IDF:** it down-weights words that appear everywhere and removes length; we tried it, and it improved log-linear by +0.02 to +0.03 macro-F1. (b) **Bigrams:** phrases like "law school" or "yoga classes" carry meaning that single words lose. Other options: averaged pretrained word embeddings, removing gendered words.
6. **One idea to improve the architecture?** Add a hidden layer, change the activation, or tune dropout. We tried a second hidden layer. A plain stack did worse than the MLP (TF-IDF's tiny numbers starved the second layer); with LayerNorm and a residual shortcut it matches the MLP but doesn't beat it, because a bag of words offers little extra structure. Another reasonable idea: weight the loss by inverse class frequency, which targets the imbalance and macro-F1 directly.

7. **How could your labelled data defend or challenge accuracy, and what would you use instead?** Look at the label distribution. AG News is balanced (4 equal classes), so accuracy is fine there. Our data is not: professor is 30%, so "always professor" gets 0.30 accuracy with near-zero usefulness (macro-F1 0.016). Use macro-F1 (or balanced accuracy), and inspect per-class F1 and the confusion matrix.